Smart HAS - Comunidade Conectada

Fase 7 - Consolidação e entrega contínua

Pacotes, triggers e SQL dinâmico em Oracle PL/SQL, e pipeline CI/CD

Integrante: Yuri Raphael Alves de Sousa
RM: 556163
FIAP - Faculdade de Informática e Administração Paulista
2026

1. Visão geral da entrega

O Smart HAS (ComunidadeConectada) conecta moradores de uma mesma região para compartilhar recursos por empréstimo, troca ou doação. Depois do aplicativo React Native, do backend Spring Boot, do painel Angular e da camada de persistência em Oracle, esta fase profissionaliza a entrega final: a regra de negócio dentro do banco ganha organização, automação e auditoria, e cada alteração de código passa a percorrer um pipeline que testa, valida e entrega sozinho.

O recurso de IA generativa do projeto (o chat ConectaIA, com o modelo Gemini) vem da Fase 5 e segue no aplicativo. Esta entrega concentra o esforço nas duas partes pedidas no enunciado.

Parte	O que foi pedido	O que foi entregue
1. Oracle PL/SQL	Agrupar procedures e functions em packages	4 pacotes (PKG_SH_LOG, PKG_SH_CALCULOS, PKG_SH_EMPRESTIMO, PKG_SH_ADMIN) no lugar de 4 functions e 6 procedures soltas
	Pelo menos uma trigger de auditoria ou automação	3 triggers: automação de status, auditoria de empréstimo e auditoria de usuário
	Dynamic SQL em procedure ou function	3 rotinas: CONTAGEM_LINHAS, PRC_VOLUMETRIA e BUSCA_RECURSOS, com defesa contra injeção
	Documentar todos os objetos	Seção 4 deste documento e database/README.md
2. CI/CD	Pipeline com Jenkins ou GitHub Actions	GitHub Actions, com 4 jobs (banco e backend, painel, app, deploy)
	Build e testes automatizados	Maven verify com 15 testes contra um Oracle Free, build do Angular e checagem de tipos do app
	Deploy do backend e integração com o Oracle	Smoke test da API empacotada, imagem Docker no GHCR e instalação do schema no Oracle de destino
	Logs e relatórios de execução	Resumo no run, relatórios Surefire, logs do banco e do deploy, e o jar, como artefatos

Links da entrega:

- Repositório: https://github.com/YuriRASousa/ComunidadeConectada_FASE5
- Pipeline (GitHub Actions): https://github.com/YuriRASousa/ComunidadeConectada_FASE5/actions/workflows/ci-cd.yml
- Vídeo (YouTube, não listado): https://youtu.be/kXHb9uTTgTw

2. Parte 1 - Consolidação em Oracle PL/SQL

2.1 Packages

Na Fase 6 o banco tinha quatro functions e seis procedures soltas. Funcionavam, mas cada uma era chamada pelo nome avulso, as regras de negócio (prazo padrão do empréstimo, dias de atraso para subir a severidade, nota inicial) estavam escritas dentro do código e não havia onde agrupar o que pertence ao mesmo assunto.

A solução seguiu o roteiro do exercício guiado de packages: declarar cada rotina na specification, levar o código para o body, testar e só então eliminar as versões soltas. A specification é o contrato público; o body guarda a implementação e pode mudar sem invalidar quem chama.

Pacote	Conteúdo	Por que existe
PKG_SH_LOG	GRAVA (transação autônoma)	Todo erro das rotinas deixa rastro, mesmo quando a operação principal sofre rollback
PKG_SH_CALCULOS	REPUTACAO, INDICE_CIRCULARIDADE, FICHA_RECURSO, DISTANCIA_KM; constantes c_reputacao_inicial e c_raio_terra_km	Cálculos usáveis dentro de SELECT, iguais para o app, o painel e o relatório
PKG_SH_EMPRESTIMO	REGISTRA_EMPRESTIMO, REGISTRA_DEVOLUCAO, ATUALIZA_REPUTACAO, DETECTA_ATRASOS; constantes c_dias_padrao (7), c_atraso_media (5) e c_atraso_alta (15)	Ciclo de vida do empréstimo com a regra no banco e os parâmetros de negócio num lugar só
PKG_SH_ADMIN	GERA_RESUMO_MENSAL (BULK COLLECT + FORALL), CONTAGEM_LINHAS, PRC_VOLUMETRIA, BUSCA_RECURSOS	Consolidação mensal e consultas em tempo real

Trecho da specification de PKG_SH_EMPRESTIMO, em que as constantes de negócio aparecem como parte do contrato:

```
CREATE OR REPLACE PACKAGE pkg_sh_emprestimo AS
    c_dias_padrao  CONSTANT PLS_INTEGER := 7;
    c_atraso_media CONSTANT PLS_INTEGER := 5;
    c_atraso_alta  CONSTANT PLS_INTEGER := 15;

    PROCEDURE registra_emprestimo (
        p_id_recurso     IN  sh_recurso.id_recurso%TYPE,
        p_id_solicitante IN  sh_usuario.id_usuario%TYPE,
        p_qt_dias        IN  NUMBER DEFAULT c_dias_padrao,
        p_id_emprestimo  OUT sh_emprestimo.id_emprestimo%TYPE,
        p_ds_retorno     OUT VARCHAR2);
    ...
END pkg_sh_emprestimo;
```

Como o código chama os pacotes: o backend usa SimpleJdbcCall com withCatalogName("PKG_SH_EMPRESTIMO") e o nome da procedure; as consultas usam a notação de pacote, por exemplo SELECT pkg_sh_calculos.reputacao(id_usuario) FROM sh_usuario. Para o banco que já estava na Fase 6 existem dois passos separados, migracao_f6_para_f7.sql (cria as tabelas novas) e remover_rotinas_avulsas.sql (elimina as rotinas soltas), de modo que o DROP só acontece depois de testar e de atualizar o backend.

2.2 Triggers

Foram criadas três triggers, uma para cada papel, sem duas triggers para o mesmo evento na mesma tabela, que é a boa prática apontada no material de estudo. Todas rodam depois da carga inicial, para a trilha de auditoria não registrar os dados de demonstração.

Trigger	Evento	O que faz
TRG_SH_EMPRESTIMO_BU	BEFORE UPDATE OF st_emprestimo, FOR EACH ROW, WHEN (NEW <> OLD)	Estado final (DEVOLVIDO, CANCELADO) não volta atrás (ORA-20301); ATIVO não volta para SOLICITADO (ORA-20302); preenche dt_retirada ao virar ATIVO e dt_devolucao ao virar DEVOLVIDO
TRG_SH_EMPRESTIMO_AUD	AFTER INSERT OR UPDATE	Grava em SH_AUDITORIA o estado antes e depois

	OR DELETE, FOR EACH ROW	(INSERTING, UPDATING, DELETING); cancelar o empréstimo devolve o item para a vitrine
TRG_SH_USUARIO_AUD	AFTER UPDATE OF ds_papel, st_usuario, fl_verificado	Audita mudança de papel (USER para ADMIN), desativação e verificação; nunca lê nem grava a senha

```
CREATE OR REPLACE TRIGGER trg_sh_emprestimo_bu
BEFORE UPDATE OF st_emprestimo ON sh_emprestimo
FOR EACH ROW
WHEN (NEW.st_emprestimo <> OLD.st_emprestimo)
BEGIN
    IF :OLD.st_emprestimo IN ('DEVOLVIDO', 'CANCELADO') THEN
        raise_application_error(-20301, 'Emprestimo ' || :OLD.id_emprestimo ||
            ' ja esta ' || :OLD.st_emprestimo || ' e nao pode mudar para ' || :NEW.st_emprestimo
|| '.');
    END IF;
    IF :NEW.st_emprestimo = 'ATIVO' AND :NEW.dt_retirada IS NULL THEN
        :NEW.dt_retirada := SYSDATE;
    END IF;
    IF :NEW.st_emprestimo = 'DEVOLVIDO' AND :NEW.dt_devolucao IS NULL THEN
        :NEW.dt_devolucao := SYSDATE;
    END IF;
END;
```

A regra vale para qualquer porta de entrada: a procedure chamada pelo app, o painel administrativo ou um UPDATE manual feito por quem tem acesso ao banco. A tabela SH_AUDITORIA participa da transação do evento, de modo que, se a alteração sofrer rollback, a linha de auditoria some junto e a trilha nunca descreve algo que não aconteceu. Já o log de PKG_SH_LOG faz o contrário, e de propósito: usa transação autônoma para sobreviver ao rollback.

Execução real no Oracle da FIAP, com um empréstimo pedido e devolvido pela procedure e uma tentativa de reabri-lo:

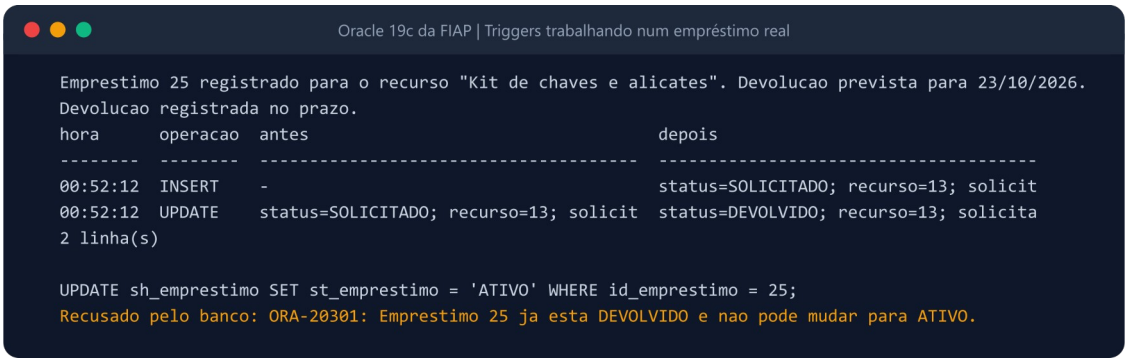


Figura 1 - A trilha de auditoria gerada pelas triggers e a recusa ORA-20301 (saída de database/demo_fase7.py).

2.3 Dynamic SQL

Três rotinas de PKG_SH_ADMIN montam comandos em tempo de execução. O texto vem de fora, então a defesa contra SQL injection faz parte do desenho: valor vai sempre por bind, nome de objeto passa pelo dicionário e por DBMS_ASSERT, e a ordenação sai de uma lista fechada.

Rotina	Para que serve	Técnica e defesa
CONTAGEM_LINHAS(tabela)	Quantas linhas tem uma tabela do schema	EXECUTE IMMEDIATE; o nome precisa existir em USER_TABLES e vai ao comando via DBMS_ASSERT.ENQUOTE_NAME. Outro nome gera ORA-20401
PRC_VOLUMETRIA	Fotografa o tamanho de todas as tabelas SH_ em SH_VOLUMETRIA, para acompanhar o crescimento do banco (e o custo na nuvem)	Laço sobre USER_TABLES com LIKE 'SH_%' ESCAPE e chamada a CONTAGEM_LINHAS
BUSCA_RECURSOS(...)	Busca de recursos por categoria, tipo, disponibilidade, texto, raio em km e ordem	OPEN v_cursor FOR v_sql USING ... com binds; só os filtros informados entram no WHERE; ordem em RECENTE, NOME ou DISTANCIA, qualquer

		outro valor gera ORA-20402; devolve SYS_REFCURSOR

```
v_order_by := CASE UPPER(NVL(p_ordem, 'RECENTE'))
                  WHEN 'RECENTE'   THEN 'dt_cadastro DESC'
                  WHEN 'NOME'      THEN 'nm_recurso'
                  WHEN 'DISTANCIA' THEN 'distancia_km NULLS LAST, nm_recurso'
                  ELSE NULL END;

v_sql := v_sql || CASE WHEN p_ds_tipo IS NOT NULL
                       THEN ' AND r.ds_tipo = :tipo'
                       ELSE ' AND :tipo IS NULL' END;
...
OPEN v_cursor FOR v_sql
    USING p_lat, p_lon, p_id_categoria, UPPER(p_ds_tipo),
          UPPER(p_ds_disponibilidade), v_texto, v_raio;
```

Os filtros ausentes viram AND :x IS NULL, o que mantém fixa a quantidade de binds sem pesar no plano de execução. O backend ganhou os endpoints GET /api/admin/recursos/busca, POST /api/admin/rotinas/volumetria e GET /api/admin/volumetria, que apenas passam os parâmetros e leem o cursor.

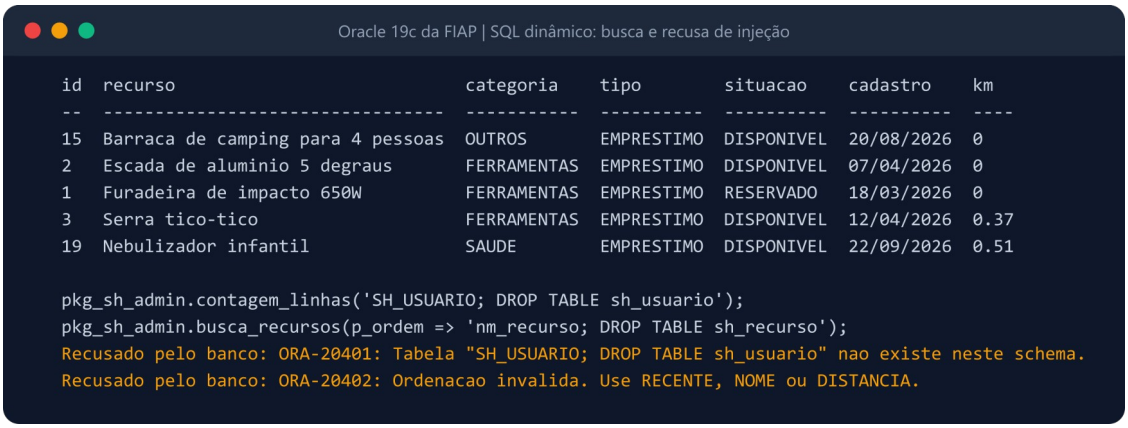


Figura 2 - Busca dinâmica por raio e recusa de duas tentativas de injeção (saída de database/demo_fase7.py).

2.4 Integração com o backend

- JdbcEmprestimoRepository e JdbcPainelRepository chamam os pacotes pelo catálogo; nenhum nome antigo sobrou no código Java.
- Os erros de aplicação dos pacotes (faixa ORA-20000 a 20999) continuam sendo traduzidos por OracleErrors e GlobalExceptionHandler, agora também os códigos 20301, 20302 (409) e 20401 (404). O morador lê a mensagem escrita dentro do banco.
- A busca por proximidade da vitrine passou a usar PKG_SH_CALCULOS.DISTANCIA_KM.

2.5 Testes

Os 14 testes de integração (PlsqlPackagesTest) rodam contra um Oracle de verdade e só são ativados quando SMARTHAS_DB_TESTS=true, o que acontece no pipeline e na máquina de quem tem o schema instalado. Cada teste cria os próprios dados e apaga tudo no final.

Teste	O que confere
`objetosEstaoValidos`	4 pacotes e 3 triggers habilitadas, nenhum objeto inválido no schema
`reputacaoInicial`, `distanciaHaversine`, `competenciaInvalida`	Nota 5 para quem não foi avaliado; São Paulo a Rio perto de 360 km; competência inválida gera ORA-20101
`cicloDeEmprestimo`	Pedido reserva o item, devolução com nota 4 libera o item e leva a reputação do dono a 4
`regrasDaProcedure`	Dono não pega o próprio item (20203), item reservado é recusado (20205), recurso inexistente (20201)
`triggerAuditaEmprestimo`	Pedido e devolução geram exatamente INSERT e UPDATE na trilha

`triggerImpedeVoltarStatus`	DEVOLVIDO para ATIVO é recusado com ORA-20301
`triggerAutomatizaDatasELiberacao`	ATIVO preenche a retirada, ATIVO para SOLICITADO é recusado (20302), CANCELADO libera o item
`triggerAuditaPapelDoUsuario`	Mudar o papel gera uma linha; trocar só a senha não gera nenhuma; o hash nunca aparece
`contagemDinamica`	Contagem bate com COUNT(*); quatro textos maliciosos recusados com ORA-20401
`volumetria`	Uma linha por tabela SH_ na coleta
`buscaDinamicaComFiltros`	Filtro por texto, sem filtro e por raio de 50 km ordenado por distância
`buscaDinamicaResisteAInjecao`	Texto com aspas vira valor e não acha nada; ordenação com DROP TABLE gera ORA-20402 e a tabela continua lá

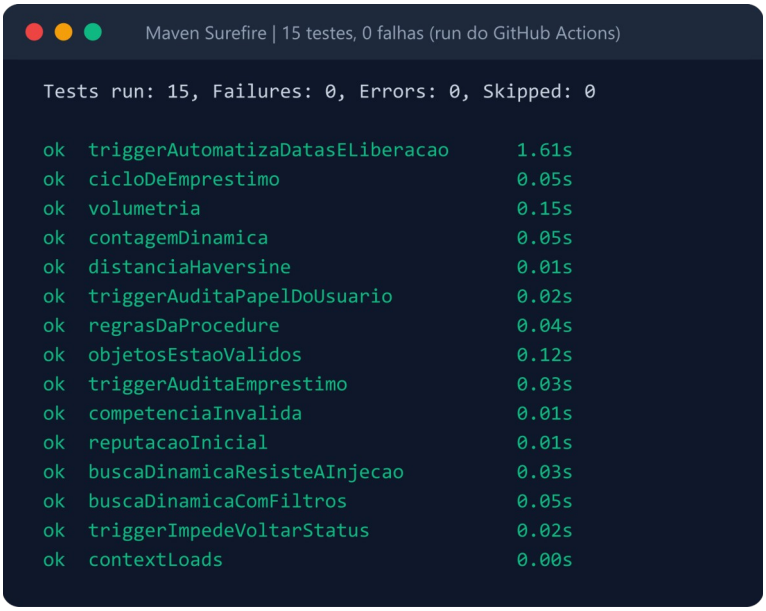


Figura 3 - Relatório do Surefire no run do GitHub Actions: 15 testes (14 de integração e o contextLoads), nenhuma falha.

3. Parte 2 - Automação e entrega contínua

3.1 Por que GitHub Actions

O repositório já está no GitHub, então o pipeline vive no mesmo lugar do código, escrito em YAML e versionado junto (.github/workflows/ci-cd.yml). Não é preciso manter um servidor próprio, como seria com o Jenkins estudado na fase, e os segredos ficam num cofre do próprio repositório. O desenho dos estágios (compilar, testar, empacotar, implantar) é o mesmo de um pipeline do Jenkins.

3.2 Desenho do pipeline

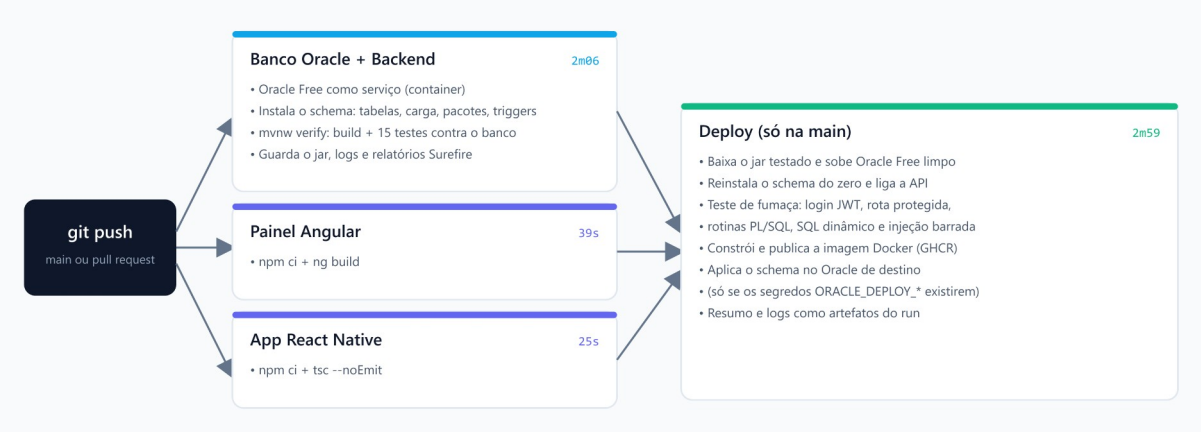


Figura 4 - Jobs do pipeline e tempos medidos no run 39094748815.

Job	Quando roda	O que faz
Banco Oracle + Backend	Todo push e pull request	Sobe o Oracle Free (gvenzl/oracle-free) como container de serviço; instala o schema com database/executar_scripts.py (tabelas, carga, pacotes, triggers) e falha se algo ficar inválido; roda mvnw verify (compila e executa os 15 testes contra o banco); publica resumo, relatórios Surefire, logs e o jar
Painel Angular	Todo push e pull request	npm ci e ng build
App React Native	Todo push e pull request	npm ci e tsc --noEmit
Deploy	Só na main, depois dos três anteriores	Baixa o jar testado, reinstala o schema num Oracle limpo, sobe a API e executa o smoke test; constrói e publica a imagem Docker no GitHub Container Registry; aplica o schema no Oracle de destino se os segredos ORACLE_DEPLOY_DSN, ORACLE_DEPLOY_USER e ORACLE_DEPLOY_PASSWORD existirem

3.3 Build e testes automatizados

O primeiro job reproduz, a cada alteração, exatamente o que o desenvolvedor faria à mão, mas num ambiente limpo: banco novo, schema novo, dependências baixadas do zero. Como o Oracle é um serviço do próprio job, os testes de integração rodam contra um banco real, e não contra um simulado. A instalação do schema também é verificada: o script termina com erro se qualquer comando falhar ou se sobrar objeto inválido em USER_OBJECTS ou USER_ERRORS.

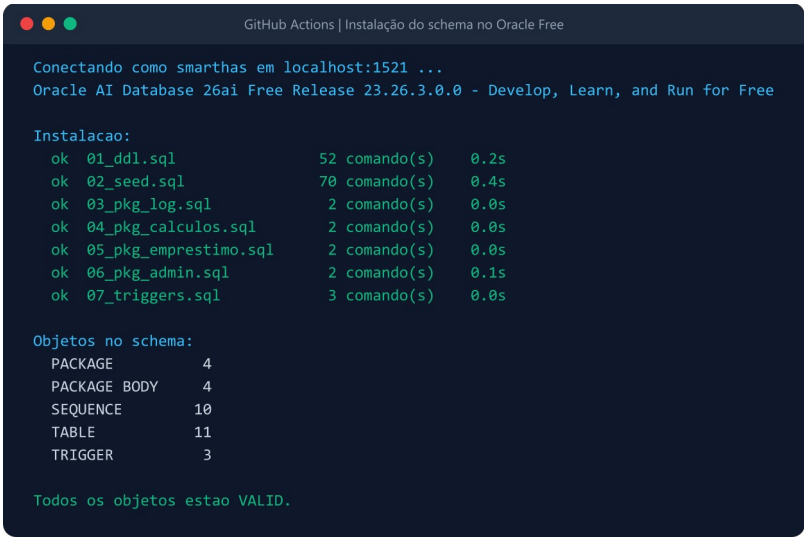


Figura 5 - Instalação do schema no Oracle Free dentro do pipeline.

3.4 Deploy automatizado e integração com o Oracle

- Smoke test (scripts/ci/smoke_test.sh): a API empacotada, ligada ao Oracle recém-instalado, responde de ponta a ponta. Confere login com JWT, recusa de rota administrativa sem token, estatísticas lidas do banco, execução de PKG_SH_EMPRESTIMO.DETECTA_ATRASOS, de PRC_VOLUMETRIA e da busca dinâmica, e a recusa de uma ordenação com injeção (HTTP 400).
- Imagem Docker: Dockerfile em duas etapas, rodando sem root, publicada em ghcr.io com a tag do commit e latest. Credenciais entram por variável de ambiente, nunca na imagem.
- Oracle de destino: com os três segredos configurados, o job aplica o schema com executar_scripts.py --migrar, que é idempotente e não recarrega os dados. Sem eles, o passo é pulado e isso aparece no resumo do run. Isso mantém o repositório seguro e permite apontar o deploy para o Oracle da FIAP ou, no futuro, para um Autonomous Database na OCI.
- docker-compose.yml reproduz Oracle e API na máquina do desenvolvedor com um comando.

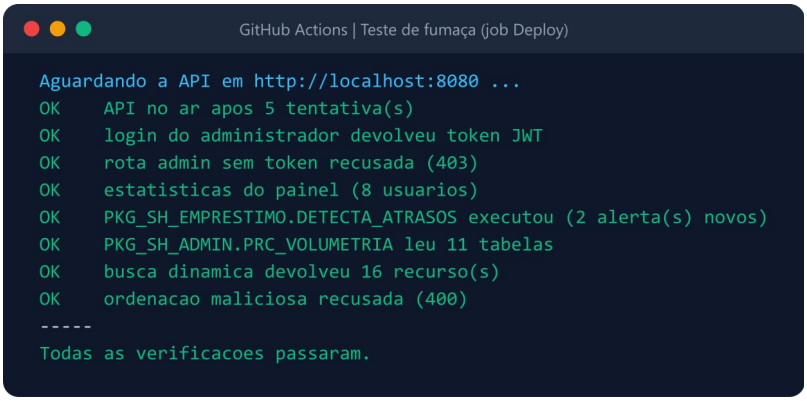


Figura 6 - Teste de fumaça executado no job de deploy.

3.5 Logs e relatórios

Artefato ou resumo	Conteúdo
Resumo do run (aba Summary)	Tabela com cada teste, resultado e tempo; últimas linhas do build do Angular e do app; resultado do deploy
logs-e-relatorios-backend	banco-instalacao.log, backend-maven.log e os XML e TXT do Surefire
smarthas-api-jar	O jar exato que foi testado, usado depois no deploy
logs-do-deploy	deploy-banco.log, api.log e smoke-test.log

3.6 Resultado da execução

O pipeline foi executado no GitHub na main e terminou com sucesso: quatro jobs verdes em 5 min 10 s no total. O Oracle do serviço era um Oracle AI Database 26ai Free (23.26.3), enquanto a FIAP usa o 19c; os scripts rodaram sem alteração nas duas versões. O passo do Oracle de destino foi pulado de forma controlada, porque nenhum segredo de deploy está configurado.

Job	Duração	Resultado
Painel Angular (build)	39 s	sucesso
App React Native (tipos)	25 s	sucesso
Banco Oracle + Backend (build e testes)	2 min 6 s	sucesso, 15 testes aprovados
Deploy (smoke test, imagem Docker e Oracle)	2 min 59 s	sucesso, 8 verificações aprovadas, imagem publicada

3.7 Segurança do pipeline

- Nenhuma credencial real no repositório. O Oracle do CI é descartável e existe só durante o job; os dados de acesso dele são conhecidos e sem valor.
- O destino real só recebe o schema quando os segredos ORACLE_DEPLOY_* existem, e eles ficam no cofre do GitHub.
- O workflow declara permissions: contents: read por padrão; só o job de deploy recebe packages: write, para publicar a imagem.
- As ações usadas estão nas versões atuais (a primeira execução acusou aviso de ações descontinuadas, corrigido em seguida).

4. Documentação dos objetos criados

Todos os objetos desta fase, com a finalidade de cada um e o valor que agrega ao sistema. A mesma lista, com o passo a passo de instalação, está em database/README.md.

4.1 Objetos Oracle

Objeto	Tipo	Finalidade	Valor agregado
PKG_SH_LOG.GRAVA	Procedure em pacote	Registrar ocorrências em SH_LOG_SISTEMA	Rastro garantido mesmo com rollback
PKG_SH_CALCULOS.REPUTACAO	Function	Média das notas recebidas (5 se não avaliado)	Mesma nota no app, painel e relatório
PKG_SH_CALCULOS.INDICE_CIRCULARIDADE	Function	Empréstimos concedidos por recurso no mês	Indicador central do projeto
PKG_SH_CALCULOS.FICHA_RECURSO	Function	Texto formatado do item	Formatação num só lugar
PKG_SH_CALCULOS.DISTANCIA_KM	Function DETERMINISTIC	Haversine entre duas coordenadas	Ordena e filtra a vitrine por proximidade
PKG_SH_EMPRESTIMO.REGISTRA_EMPRESTIMO	Procedure	Validar e registrar o pedido	Regra única para todas as portas de entrada
PKG_SH_EMPRESTIMO.REGISTRA_DEVOLUCAO	Procedure	Fechar o ciclo, avaliar e recalcular reputação	Consistência entre empréstimo, item e reputação
PKG_SH_EMPRESTIMO.ATUALIZA_REPUTACAO	Procedure	Gravar a reputação em SH_USUARIO	App não recalcula a média a cada tela
PKG_SH_EMPRESTIMO.DETECTA_ATRASOS	Procedure	Abrir alerta para empréstimo vencido	Vigilância sem repetir alerta
PKG_SH_ADMIN.GERA_RESUMO_MENSAL	Procedure	Recarregar SH_RESUMO_USUARIO_MES	Painel lê números prontos; BULK COLLECT e FORALL
PKG_SH_ADMIN.CONTAGEM_LINHAS	Function (SQL dinâmico)	Contar linhas de uma tabela	Base da volumetria, com defesa contra injeção
PKG_SH_ADMIN.PRC_VOLUMETRIA	Procedure (SQL dinâmico)	Fotografar o tamanho das tabelas	Acompanhar crescimento e custo na nuvem
PKG_SH_ADMIN.BUSCA_RECURSOS	Function (SQL dinâmico)	Busca com filtros opcionais	Consulta flexível em tempo real, sem concatenar valores
TRG_SH_EMPRESTIMO_BU	Trigger	Regras e datas do status do empréstimo	Estado inválido barrado na origem
TRG_SH_EMPRESTIMO_AUD	Trigger	Auditar e liberar o item cancelado	Rastreabilidade e automação
TRG_SH_USUARIO_AUD	Trigger	Auditar papel, status e verificação	Segurança e LGPD, sem guardar senha
SH_AUDITORIA	Tabela	Trilha de auditoria	Quem mudou o quê, antes e depois
SH_VOLUMETRIA	Tabela	Histórico do tamanho das tabelas	Planejamento de capacidade
sq_sh_auditoria, sq_sh_volumetria	Sequences	Chaves das duas tabelas novas	Sem MAX+1

4.2 Outros arquivos

Arquivo	Finalidade
database/executar_scripts.py	Instala o schema na ordem certa e confere se tudo ficou válido; usado à mão e no pipeline
database/migracao_f6_para_f7.sql, remover_rotinas_avulsas.sql	Passagem do banco da Fase 6 para os pacotes, em dois passos
database/demo_fase7.py	Demonstração para o vídeo; apaga o que cria
backend/.../db/PlsqlPackagesTest.java	14 testes de integração contra o Oracle
.github/workflows/ci-cd.yml	Pipeline
scripts/ci/smoke_test.sh, resumo_testes.py	Teste de fumaça e resumo dos testes
backend/smarthas-api/Dockerfile, docker-	Imagem e ambiente local

`compose.yml`	

5. Como reproduzir

```
# banco (Oracle 19c ou 23ai)
pip install oracledb
python database/executar_scripts.py          # banco vazio
python database/executar_scripts.py --migrar # banco que estava na Fase 6

# backend e testes
cd backend/smarthas-api
SMARTHAS_DB_TESTS=true ./mvnw verify

# ambiente completo com Docker
docker compose up --build
```

6. Problemas encontrados e decisões

- A exportação do deck de slides para PDF fechava o LibreOffice quando havia cartões altos empilhados com sombra; a solução foi remover as sombras dos cartões, sem perda de leitura.
- Ao validar o smoke test na máquina local, a rotina de atrasos já não abria alertas novos, porque estavam abertos desde antes. A verificação passou a conferir que a rotina executa, e não que abre pelo menos um alerta; o banco limpo do pipeline continua abrindo os dois da carga inicial.
- O texto de apoio da fase traz trechos de código com falhas (colunas fora de ordem, variáveis a menos num FETCH). Foram reescritos e testados em vez de copiados.
- Decisão: as rotinas soltas da Fase 6 só são eliminadas por um script à parte, depois dos testes, para o banco nunca ficar sem as duas versões ao mesmo tempo.

7. Entregáveis

- Este relatório em PDF.
- Apresentação de slides em PDF, com 10 slides e o grupo no primeiro.
- Código no GitHub: https://github.com/YuriRASousa/ComunidadeConectada_FASE5
- Vídeo de até 5 minutos no YouTube (não listado): https://youtu.be/kXHb9uTTgTw